Lab 5: Asynchronous operation
Geavanceerde Computerarchitectuur: Labosessie

(B-KUL-JLI1QS)

Timon Coen

March 24, 2024



Group: MEIswe

Instructor: Alex Marinšek

Introduction

In this lab session, we were familiarised with dynamic parallelism and asynchronous operations. This again with the goal of optimising the execution time and GPU usage. Due to the limitations of the GPUs in the lab, which don't support dynamic parallelism, it was only possible to test and explore asynchronous operations.
The goal of this lab is to find the difference between asynchronous and synchronous operations. Asynchronous operations can be executed in parallel, independently of each other. Like this, the CPU can do other things in stead of waiting.

Assignment

The assignment for this lab is to call 4 different reduction kernels and letting the CPU do other things while the GPU is executing the kernel and copying memory. Different operations that are completely separate from each other can be put in different streams, resulting in different GPU call stack. The measurements were done on the GT 940 with an array of 4 to an array of 2048, without striding. In figure 1, you can clearly see asynchronous operations have a significant improvement. The program with asynchronous calls can execute instructions on the CPU while the GPU is busy. You can also give flags to the separate streams, giving a certain priority if needed. For these executions I used `cudaStreamNonBlocking` for each stream.

Conclusion

If multiple different things need to be processed on the GPU, making use of asynchronous operations can give a significant time improvement, because the CPU can continue executing while the GPU is executing its kernel code.

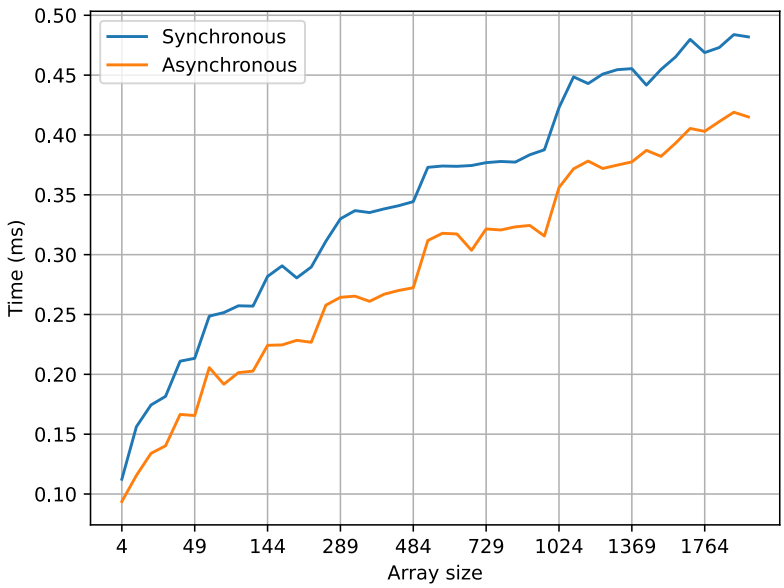


Figure 1: Execution time per array size for synchronised and asynchronised